

BANCO DE DADOS - CONTROLE DE CONCORRÊNCIA

Profa. Vandecia Fernandes – BiCT/ECP/UFMA

Referências: Elmasri, R. and Navathe, S.B. Sistemas de Bancos de Dados. Korth, H.F. e Silberschatz, A. Sistemas de Bancos de Dados. Notas de aula dos professores: Clodis Boscarioli e Frank A. Siqueira



CONTROLE DE CONCORRÊNCIA

- Uma das propriedades fundamentais de uma transação é o isolamento.
- Quando diversas transações são executadas de modo concorrente corre-se o risco de violar esta propriedade.

CONTROLE DE CONCORRÊNCIA

- Tipos de anomalias causadas pela falta de isolamento entre transações:
 - perda de consistência do banco
 - perda de atualizações
- Controle de concorrência: necessário
 - propriedade de serialização

CONTROLE DE CONCORRÊNCIA

- Protocolos de controle garantem a serialização no processamento de transações.

CONTROLE DE CONCORRÊNCIA

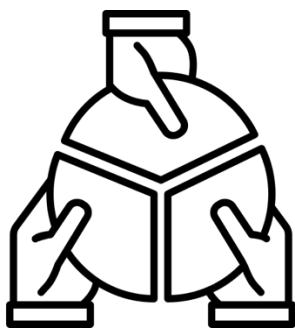
- Protocolos usados para controlar a concorrência:
 - Protocolos de bloqueio
 - Protocolos com base em timestamps
 - Protocolos de validação
 - Protocolos multipl versão

PROTOCOLOS DE BLOQUEIO

-
- Impedem que um dado seja modificado enquanto uma transação o estiver acessando.

PROTOCOLO DE BLOQUEIO

■ Modos de bloqueio



Compartilhado (S):
permite somente leitura;
obtido sempre que não
houver nenhum bloqueio
exclusivo.



Exclusivo (X):
permite leitura e escrita;
obtido somente se não houver
nenhum outro bloqueio.

PROTOCOLOS DE BLOQUEIO

- Todas as transações devem:
 1. Solicitar o bloqueio compartilhado (para leitura) ou exclusivo (para escrita) antes de acessar um dado
 - Autorização (*grant*) vai depender dos bloqueios existentes
 2. Liberar o bloqueio quando não for mais necessário
 - Uma transação pode desbloquear um item de dado a qualquer momento, mas deverá manter o bloqueio enquanto quiser operar sobre o dado.

PROTOCOLOS DE BLOQUEIO

- Operações de bloqueio:

Operação	Ação
lock-S(dado)	Solicita bloqueio compartilhado do dado
lock-X(dado)	Solicita bloqueio exclusivo do dado
unlock(dado)	Libera o bloqueio existente

- Transações em espera ganham direito de acesso quando o dado bloqueado for liberado.

PROTOCOLOS DE BLOQUEIO

- Considere as transações abaixo:

T_1 :

```
lock-X(B);  
read(B);  
B := B - 50;  
write(B);  
unlock(B);  
lock-X(A);  
read(A);  
A := A + 50;  
write(A);  
unlock(A);
```

T_2 :

```
lock-S(A);  
read(A);  
unlock(A);  
lock-S(B);  
read(B);  
display (A + B);  
unlock(B);
```

Se $A = 100$ e $B = 200$, o valor mostrado em T_2 = 300.

PROTOCOLOS DE BLOQUEIO

T ₁	T ₂	Controle de concorrência
lock-X(B)		
read(B)		grant-X(B ₁ , T ₁)
B := B - 50		
write(B);		
unlock(B)		
	lock-S(A)	
	read(A)	grant-S(A ₁ , T ₂)
	unlock(A)	
	lock-S(B)	
	read(B)	grant-S(B ₁ , T ₂)
	display(A+B)	
	unlock(B)	
lock-X(A)		
read(A)		grant-X(A ₂ , T ₁)
A := A + 50		
write(A)		
unlock(A)		

PROTOCOLOS DE BLOQUEIO

- A escala de execução mostrada no slide anterior não é serializável em conflito e não é serializável em visão.
- A transação T2 mostra o valor 250 dólares.
- A forma como os bloqueios foram concedidos permitiu que este escalonamento fosse criado.
- O controlador de concorrência pode impedir que escalonamentos errados sejam criados concedendo bloqueios de uma forma mais restrita.

PROTOCOLOS DE BLOQUEIO

- Protocolo de bloqueio em duas fases (two phase locking – 2PL):
 - Esse protocolo exige que cada transação emita suas solicitações de bloqueio e desbloqueio em duas fases:
 - Fase de expansão
 - Fase de encolhimento

PROTOCOLOS DE BLOQUEIO

1. Primeira fase: Fase de expansão

- uma transação pode obter bloqueios, mas não pode liberar nenhum.

2. Segunda fase: Fase de encolhimento

- uma transação pode liberar bloqueios, mas não consegue obter nenhum novo bloqueio.

PROTOCOLOS DE BLOQUEIO

- Bloqueio pode causar deadlock e starvation
- Deadlock: Ocorre quando cada transação em um conjunto de duas ou mais transações espera por algum item que esteja bloqueado por alguma transação no conjunto.
- Starvation(inanição): Ocorre quando uma transação não pode continuar por um período indefinido de tempo, enquanto outras transações continuam normalmente.

PROTOCOLOS DE BLOQUEIO

- Protocolo de bloqueio em duas fases:
 - Garante escala de execução serializável em conflito
 - A Precedência é determinada em função do instante de obtenção do último bloqueio.

PROTOCOLOS DE BLOQUEIO

- Exemplo de bloqueio em duas fases

T_1	T_2
Lock-X (Aplic); Read (Aplic); Aplic.Saldo = Aplic.Saldo - 500; Write (Aplic); Lock-X (Conta); Read (Conta);	
Bloqueada	Lock-S (Conta); // Bloqueio
Conta.Saldo = Conta.Saldo + 500; Write (Conta); Unlock (Conta); Unlock (Aplic);	Bloqueada
	Read (Conta); Lock-S (Aplic); Read (Aplic); Print (Conta.Saldo + Aplic.Saldo); Unlock (Conta); Unlock (Aplic);

PROTOCOLOS DE BLOQUEIO

- Bloqueio em duas fases com deadlock:

T ₁	T ₂
Lock-X (Aplic); Read (Aplic); Aplic.Saldo = Aplic.Saldo - 500; Write (Aplic);	
Bloqueada	Lock-S (Conta); Read (Conta); Lock-S (Aplic);
Lock-X (Conta);	Bloqueada
Bloqueada	Bloqueada
Não executa: Read (Conta); Conta.Saldo = Conta.Saldo + 500; Write (Conta); Unlock (Aplic); Unlock (Conta);	Não executa: Read (Aplic); Print (Conta.Saldo + Aplic.Saldo); Unlock (Conta); Unlock (Aplic);

PROTOCOLOS DE BLOQUEIO

- Variantes do bloqueio em duas fases:
 - Evitam o rollback em cascata
- Protocolo de bloqueio em duas fases severo
- Protocolo de bloqueio em duas fases rigoroso

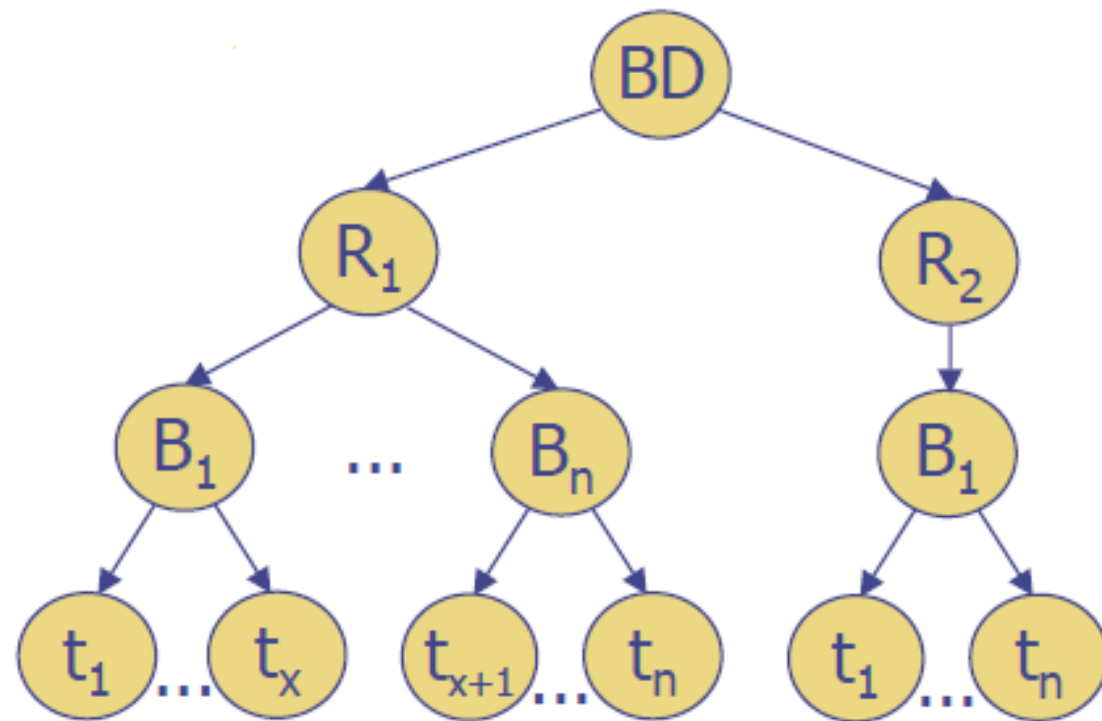
PROTOCOLOS DE BLOQUEIO

- Protocolo de bloqueio em duas fases severo:
 - Obriga que os bloqueios exclusivos sejam mantidos até a efetivação da transação
- Protocolo de bloqueio em duas fases rigoroso:
 - Obriga que todos os bloqueios (compartilhados e exclusivos) sejam mantidos até o commit

PROTOCOLOS DE BLOQUEIO

- Granularidade Múltipla:
 - Ao invés de bloquear um item de dados, podemos bloquear tuplas, tabelas, blocos de disco ou BDs
 - Podemos usar árvores para escolher a granularidade do bloqueio
 - Cada nó da árvore pode ser bloqueado individualmente
 - Cada nível corresponde a uma granularidade de bloqueio

PROTOCOLOS DE BLOQUEIO



PROTOCOLOS DE BLOQUEIO

- Protocolo de bloqueio baseados em grafos:
 - Determinam uma ordenação parcial no acesso aos dados usando um grafo de precedência

PROTOCOLOS DE BLOQUEIO

- Protocolo de bloqueio baseados em grafos:
 - Supondo que exista a relação de precedência $A \rightarrow B$ no grafo, qualquer transação que acesse A e B, deve acessar primeiro A e depois B
 - A ordenação pode ser baseada na organização física ou lógica dos dados, ou pode ser imposta de modo aleatório somente para controlar a concorrência
 - Existem vários protocolos baseados em grafos; um dos mais simples é o protocolo de grafo em árvore

PROTOCOLOS DE BLOQUEIO BASEADOS EM GRAFOS

- Protocolo de grafo em árvore
 - Bloqueios são todos exclusivos
 - Primeiro bloqueio pode ser em qualquer dado
 - Os bloqueios seguintes podem ocorrer somente se o dado precedente estiver bloqueado

PROTOCOLOS DE BLOQUEIO

■ Protocolo de grafo em árvore

VANTAGENS

- Dados podem ser desbloqueados a qualquer instante
- Bloqueio mais curto que no protocolo de 2 fases
- Garante serialização de conflito e evita deadlock

DESVANTAGENS

- Pode bloquear mais dados do que realmente precisa
- Escalas serializáveis por este protocolo podem não o ser com bloqueio em 2 fases, e vice-versa

PROTOCOLOS COM BASE EM TIMESTAMPS

-
- Associam timestamps (marcas de tempo) às transações e aos dados para ordenar operações de leitura e escrita

PROTOCOLO COM BASE EM TIMESTAMPS

- Timestamp: identificador único criado pelo SGBD para identificar uma transação. São associados na ordem em que as transações são submetidas ao sistema.
 - $TS(T)$ = timestamp da transação T.
- Garante seriabilidade: No escalonamento, as transações tomam a ordem de seus timestamps

PROTOCOLO COM BASE EM TIMESTAMPS

- Dois timestamps são associados a cada dado:
 - **W-TS**: indica a transação de maior timestamp que alterou o valor do dado
 - **R-TS**: indica a transação e maior timestamp que leu o valor do dado
- Esses timestamps são atualizados sempre que uma instrução **read(X)** ou **write(X)** é executada.
- Assegura que as operações de leitura e escrita sejam executadas por ordem de timestamp.

PROTOCOLO COM BASE EM TIMESTAMPS

- Ordenação de operações por timestamps:

LEITURA

- No caso da transação T querer ler um dado Q:
 - Se $TS(T) < W-TS(Q)$: T é abortada e desfeita (T quer ler um dado que já foi sobrescrito)
 - Se $TS(T) \geq W-TS(Q)$: a operação é executada;
 - Se $R-TS(Q) < TS(T)$: então $R-TS(Q) = TS(T)$

PROTOCOLO COM BASE EM TIMESTAMPS

- Ordenação de operações por timestamps:

ESCRITA

- No caso da transação T querer alterar um dado Q:
 - Se $TS(T) < R-TS(Q)$: T é abortada e desfeita (T não pode alterar um valor que já foi lido)
 - Se $TS(T) < W-TS(Q)$: T é abortada e desfeita (T está tentando escrever um valor obsoleto)
 - Senão, a operação é executada; $W-TS(Q) = TS(T)$

PROTOCOLO COM BASE EM TIMESTAMPS

- Exemplo de escala ordenada por timestamps:

T_1	T_2	Timestamps
Read (Aplic); Aplic.Saldo -= 500 Write (Aplic);		TS(T_1) = 1 R-TS(Aplic) = 1 W-TS(Aplic) = 1
Bloqueada	Read (Conta); Read (Aplic); Print (Conta.Saldo + Aplic.Saldo);	TS(T_2) = 2 R-TS(Conta) = 2 R-TS(Aplic) = 2
Read (Conta); Conta.Saldo += 500; Write (Conta);		R-TS(Conta) = 2 TS(T_1) < R-TS(Conta) → T_1 deve ser refeita
Read (Aplic); Aplic.Saldo -= 500 Write (Aplic); Read (Conta); Conta.Saldo += 500; Write (Conta);		Reinicia com TS(T_1) = 3 R-TS(Aplic) = 3 W-TS(Aplic) = 3 R-TS(Conta) = 3 W-TS(Conta) = 3

PROTOCOLO COM BASE EM TIMESTAMPS



Regra de Thomas:

Modificação no algoritmo básico, que faz com que haja menos operações de escrita, aumentando as possibilidades de concorrência.

- Se a transação T tentar alterar um dado Q , e $TS(T)$ for menor que $W-TS(Q)$, não é preciso abortar a transação, basta ignorar a operação de escrita
- Garante a serialização de visão

PROTOCOLOS BASEADOS EM VALIDAÇÃO

- Protocolo otimista de controle de concorrência.
Gerencia o acesso concorrente a dados sem usar bloqueios (locks).

PROTOCOLOS BASEADOS EM VALIDAÇÃO

- Possui 3 fases principais:

Fase de leitura (Read Phase):

- A transação executa normalmente, lendo e modificando cópias locais dos dados.
- Nenhuma escrita no banco de dados real é feita ainda.

Fase de validação (Validation Phase):

- Quando a transação termina e está pronta para ser confirmada (commit), o sistema verifica se **houve conflito com outras transações concorrentes**.
- Verifica-se, por exemplo, se outra transação escreveu em algum dado que essa transação leu.

Fase de escrita (Write Phase):

- Se a transação **passar na validação**, suas alterações são aplicadas no banco de dados.
- Se **falhar na validação**, ela é **abortada** e geralmente reiniciada.

PROTOCOLOS BASEADOS EM VALIDAÇÃO

- Para realizar o teste de validação, cada transação T possui três timestamps:
 - $\text{Start}(T)$: início da execução
 - $\text{Validation}(T)$: início da fase de validação
 - $\text{Finish}(T)$: final da transação

PROTOCOLOS BASEADOS EM VALIDAÇÃO

- A ordem de serialização é dada pelo momento de validação.
- Se as transações forem concorrentes, a serialização ocorre na ordem dos tempos de validação caso:
 - Dados escritos por T_i não tenham sido lidos por T_j (a escrita de T_i não invalida o valor lido por T_j)
 - Tenhamos $\text{Finish}(T_i) < \text{Validation}(T_j)$ (T_j não pode invalidar os dados lidos por T_i)
- Caso contrário, a transação é abortada e desfeita

PROTOCOLOS BASEADOS EM VALIDAÇÃO

- Se $\text{Finish}(T_i) < \text{Start}(T_j)$, as transações não são concorrentes, e a serialização certamente ocorre
- Garante serialização e evita rollback em cascata

PROTOCOLOS DE MÚLTIPLA VERSÃO

- Em vez de modificar diretamente os dados existentes, o sistema mantém múltiplas versões de cada dado.

PROTOCOLOS DE MÚLTIPLA VERSÃO

- Permite que um dado tenha vários valores (Operações de escrita criam novas versões do dado)
- Quando é feita a leitura, o SGBD escolhe a versão do dado que será lida de modo a garantir a serialização
- Operações de leitura não precisam aguardar, pois sempre há uma versão do dado pronta para ser lida
- Protocolo determina como as versões são usadas

PROTOCOLOS DE MÚLTIPLA VERSÃO

- Protocolos com base em múltiplas versões:
 - Multi-versão com ordenação por timestamps
 - Multi-versão com bloqueio em duas fases
- Garantem a criação de escalas serializáveis

PROTOCOLOS DE MÚLTIPLA VERSÃO

- Multi-versão com ordenação por timestamps:
 - Cada versão do dado possui timestamps de leitura (R-TS) e escrita (W-TS), além do valor do dado
 - Uma transação T sempre acessa a versão Q_k do dado com o maior W-TS que seja menor ou igual a $TS(T)$

PROTOCOLOS DE MÚLTIPLA VERSÃO

- Multi-versão com ordenação por timestamps:
 - A transação T sempre lê a versão Q_k
 - Ao escrever, T é desfeita se $TS(T) < R-TS(Q_k)$;
 - senão, se $TS(T) = W-TS(Q_k)$, o valor de Q_k é alterado;
 - caso contrário, uma nova versão de Q é criada
 - Versões antigas, com $W-TS(Q_k) < TS(T)$, sendo T a última transação executada, podem ser removidas

PROTOCOLOS MULTI-VERSÃO

- Exemplo de escala multi-versão com timestamps:

T_1	T_2	Timestamps
Read (Aplic); Aplic.Saldo -= 500 Write (Aplic);		$TS(T_1) = 1$ $R-TS(Aplic_1) = 1$ $W-TS(Aplic_2) = 1$
Bloqueada	Read (Conta); Read (Aplic); Print (Conta.Saldo + Aplic.Saldo);	$TS(T_2) = 2$ $R-TS(Conta_1) = 2$ $R-TS(Aplic_2) = 2$
Read (Conta); Conta.Saldo += 500; Write (Conta);		$R-TS(Conta_1) = 2$ $R-TS(Conta_1) > TS(T_1)$ $\rightarrow T_1 \text{ e } T_2 \text{ refeitas}$

PROTOCOLOS DE MÚLTIPLA VERSÃO

- Multi-versão com bloqueio em duas fases:
 - Usa timestamps e contador de commits (ts-counter).
 - Distingue transações de atualização e somente-leitura.

PROTOCOLOS DE MÚLTIPLA VERSÃO

- Multi-versão com bloqueio em duas fases:
- Transação de atualização (update)
 - Faz o bloqueio em duas fases
 - Cada **write** cria uma nova versão Q_k do dado quando fizer o commit com $TS(Q_k) = ts-counter$

PROTOCOLOS DE MÚLTIPLA VERSÃO

- Multi-versão com bloqueio em duas fases:
- Transação somente-leitura (read-only)
 - $TS(T)$ = ts-counter no momento que ela se inicia
 - Lê a versão do dado com o maior $TS(Q_k) \leq TS(T)$

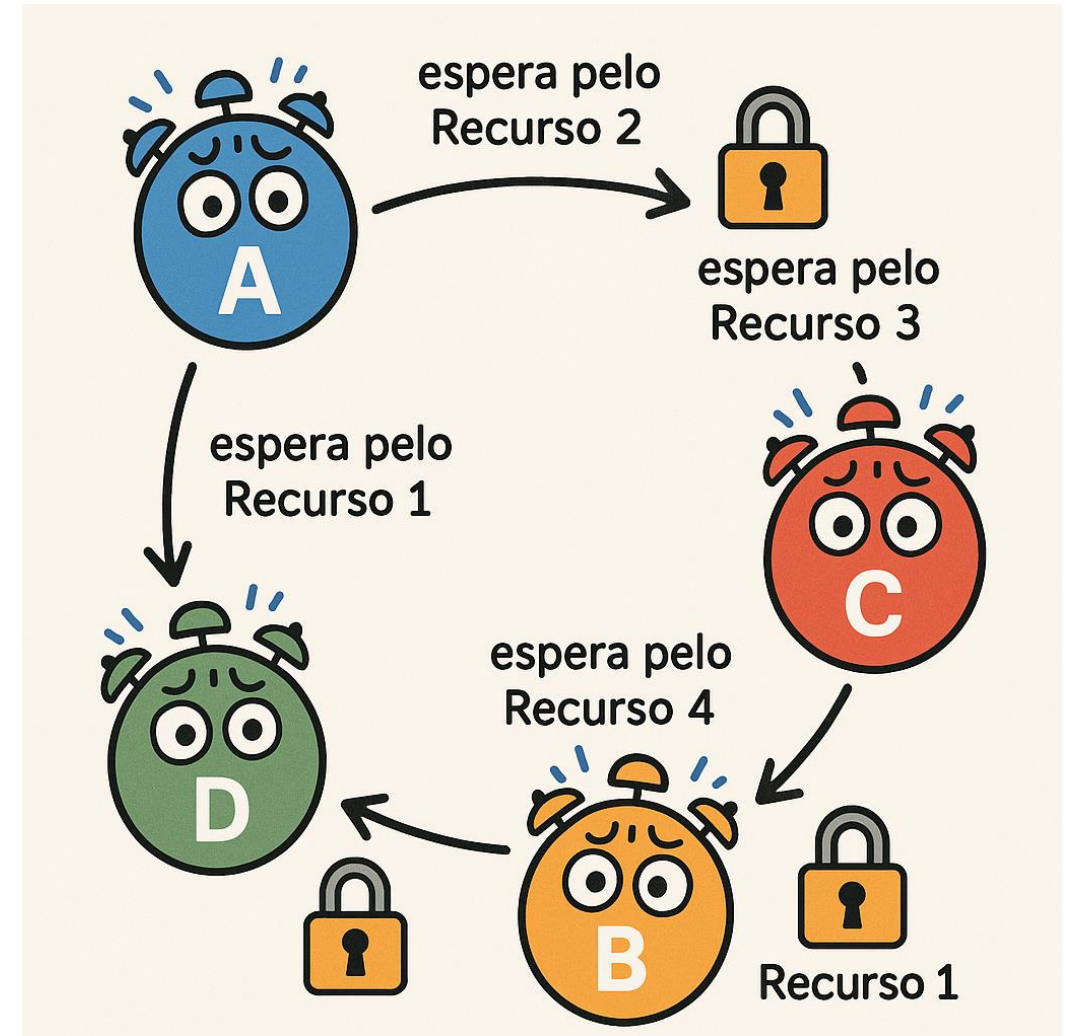
PROTOCOLOS DE MÚLTIPLA VERSÃO

- Exemplo de escala multi-versão com bloqueio em duas fases:

T_1	T_2	Timestamps
Lock-X (Aplic); Read (Aplic); Aplic.Saldo -= 500 Write (Aplic);		ts-counter = 1 → Lê Aplic ₁ = 1000 TS(Aplic ₂) = 2
Bloqueada	Lock-S (Conta); Read (Conta); Unlock (Conta);	TS(T ₂) = ts-counter = 1 → Lê Conta ₁ = 1000
Lock-X (Conta); Read (Conta); Conta.Saldo += 500; Write (Conta); Unlock (Conta); Unlock (Aplic);	Bloqueada	→ Lê Conta ₁ = 1000 TS(Conta ₂) = 2 ts-counter = 2
	Lock-S (Aplic); Read (Aplic); Print (Conta+Aplic); Unlock (Aplic);	→ Lê Aplic ₁ = 1000

CONTROLE DE DEADLOCK

- O Deadlock ocorre quando temos um conjunto de transações no qual todas estão em espera, uma aguardando o término da outra para prosseguir



CONTROLE DE DEADLOCK

- Dois métodos principais:
 - Prevenção de deadlock
 - Detecção e recuperação de deadlock

CONTROLE DE DEADLOCK - PREVENÇÃO

- **Prevenção de deadlock:**
 - garante que o sistema nunca entrará em tal situação. Mais utilizado quando há alta probabilidade do sistema entrar em deadlock .
- **Técnicas:**
 - Prevenção de deadlock usando esperar-morrer
 - Prevenção de deadlock usando ferir-esperar
 - Prevenção de deadlock usando timeout

CONTROLE DE DEADLOCK

- Prevenção de deadlock usando **esperar-morrer**:
 - Uma transação T_i somente espera um dado mantido por T_j se esta for mais nova; caso contrário T_i aborta
- Prevenção de deadlock usando **ferir-esperar**:
 - Uma transação T_i somente espera um dado mantido por T_j se esta for mais antiga; caso contrário ela obriga T_j a abortar e a liberar o dado (T_i fere T_j)

CONTROLE DE DEADLOCK

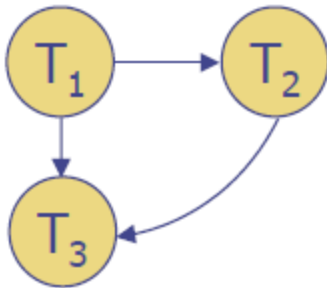
- Prevenção de deadlock usando **timeout**:
 - Pedidos de bloqueio possuem um tempo máximo de espera; se o dado não for liberado neste tempo, a transação é abortada e reiniciada

CONTROLE DE DEADLOCK - DETECÇÃO

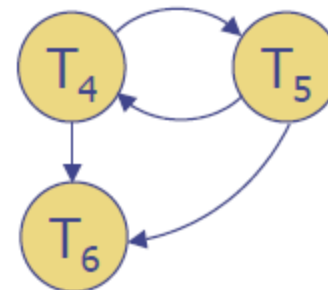
- Detecção e recuperação de deadlock: Nesse caso o sistema permite que o deadlock ocorra, mas monitora o sistema para detectá-lo e depois tenta recuperá-lo.
- Detecção:
 - Monitoramento de Estado do Sistema
 - Algoritmo de Detecção

CONTROLE DE DEADLOCK - DETECÇÃO

- Algoritmo verifica o estado do sistema periodicamente para determinar se transações estão em deadlock
- O sistema precisa manter informações sobre a alocação dos dados e as solicitações pendentes
- Deadlocks podem ser detectados usando grafos de espera, nos quais um ciclo indica um deadlock



Grafo sem ciclo



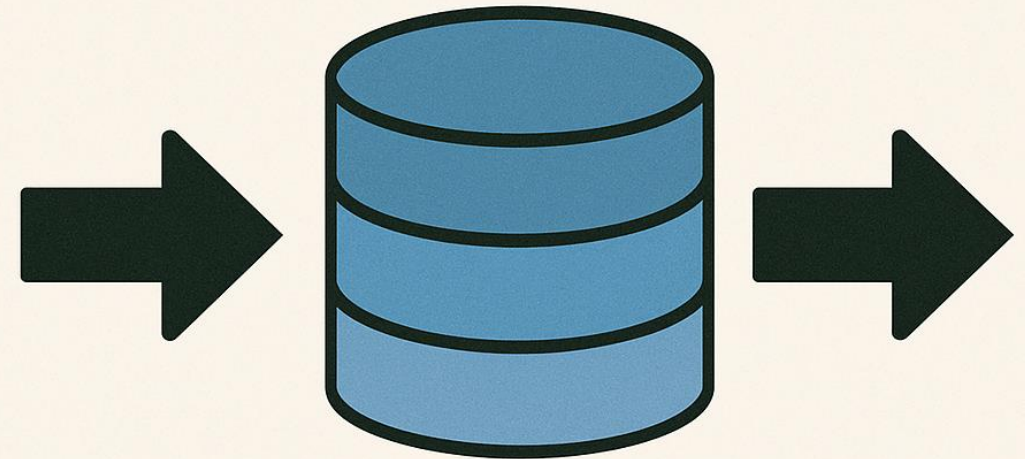
Grafo com ciclo

CONTROLE DE DEADLOCK - RECUPERAÇÃO

- Recuperação de deadlock:
 - Após detectar o deadlock, é preciso abortar uma transação para quebrar o ciclo de espera
- A escolha da transação é feita em função do custo do rollback

INSERÇÃO / REMOÇÃO DE DADOS

- Operações de inserção e remoção devem ser consideradas no controle de concorrência, pois são conflitantes com qualquer outra operação



INSERÇÃO E REMOÇÃO DE DADOS

- Técnicas:
- Protocolo de bloqueio em duas fases:
 - Devemos bloquear o dado em modo exclusivo para inserir ou remover
- Protocolo com base em timestamps:
 - Devemos inicializar $W-TS(Q)$ e $R-TS(Q)$ com o timestamp da transação que insere o dado Q



DÚVIDAS?